

PRACTICAL WEEK 4

Task-1 Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the both functions

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    // Create a new process
    pid = fork();

    if (pid < 0) {
        // Error handling if fork fails
        perror("Fork failed");
        exit(1);
    }
    else if (pid == 0) {
        // --- Child Process ---
        printf("--> [Child Process]\n");
        printf("    Child PID: %d\n", getpid());
        printf("    Parent PID: %d\n", getppid());

        // Define an argument list for the command you want to execute
        // Example: executing "ls -l"
        char *args[] = {"ls", "-l", NULL};

        printf("    Executing 'ls -l' using execvp()...\n\n");

        // execvp replaces the current child process image with a new process image
        execvp(args[0], args);

        // If execvp returns, an error occurred
        perror("execvp failed");
        exit(1);
    }
    else {
        // --- Parent Process ---
        printf("--> [Parent Process]\n");
        printf("    Parent PID: %d\n", getpid());
        printf("    Child PID created: %d\n", pid);

        // Parent waits for the child process to complete execution
        int status;
        wait(&status);

        printf("\n--> [Parent Process] Child process has finished. Parent exiting.\n");
    }

    return 0;
}
```

```

imvarls@LAPTOP-VQ4D70L7: ~$ nano process.c
imvarls@LAPTOP-VQ4D70L7: ~$ gcc process.c -o process
imvarls@LAPTOP-VQ4D70L7: ~$ ./process
--> [Parent Process]
Parent PID: 605
Child PID created: 606
--> [Child Process]
Child PID: 606
Parent PID: 605
Executing 'ls -l' using execvp()...

total 316
-rwxr-xr-x  1 imvarls imvarls 16256 Aug  4 07:11 File.txt
-rw-r--r--  1 imvarls imvarls   78 Aug  4 15:19 File1.tx
-rw-r--r--  1 imvarls imvarls   54 Aug  4 15:20 File1.txt
-rw-r--r--  1 imvarls imvarls  1308 Aug  4 07:11 File1.txt.c
drwxr-xr-x 11 imvarls imvarls  4096 Jul 30 09:22 OSSP
-rwxr-xr-x  1 imvarls imvarls 16216 Sep 11 09:22 client
-rw-r--r--  1 imvarls imvarls   719 Sep 11 09:20 client.c
-rwxr-xr-x  1 imvarls imvarls 16200 Aug  4 15:20 copy
-rw-r--r--  1 imvarls imvarls   820 Aug  4 15:20 copy.c
-rw-r--r--  1 imvarls imvarls  1529 Aug  4 14:53 copy_file.c
-rwxr-xr-x  1 imvarls imvarls 16120 Aug  8 03:57 demo
-rw-r--r--  1 imvarls imvarls   473 Aug  8 03:57 demo.c
-rw-r--r--  1 imvarls imvarls  2251 Aug 11 06:39 file.c
-rw-r--r--  1 imvarls imvarls   104 Aug 11 06:44 file.java
-rw-r--r--  1 imvarls imvarls  1536 Aug  8 03:24 file2.c
-rwxr-xr-x  1 imvarls imvarls 16224 Aug  4 15:01 filename
-rw-r--r--  1 imvarls imvarls   944 Aug  4 15:01 filename.c
-rw-----  1 imvarls imvarls     6 Jul 25 04:19 filename.c.save
-rw-----  1 imvarls imvarls    23 Jul 28 06:54 first.c.save
-rw-----  1 imvarls imvarls   144 Jul 28 07:09 first.c.save.1
-rwxr-xr-x  1 imvarls imvarls 16968 Aug 11 15:19 myshell
-rw-r--r--  1 imvarls imvarls  5896 Aug 11 15:18 myshell.c
-rwxr-xr-x  1 imvarls imvarls 16344 Sep 12 02:51 process
-rw-r--r--  1 imvarls imvarls  1399 Sep 12 02:51 process.c
-rwxr-xr-x  1 imvarls imvarls 16296 Aug  8 03:34 process_state
-rw-r--r--  1 imvarls imvarls  1536 Aug  8 03:34 process_state.c
-rw-r--r--  1 imvarls imvarls  1536 Aug  8 03:18 processstate.c
-rwxr-xr-x  1 imvarls imvarls 16216 Sep 11 09:20 server
-rw-r--r--  1 imvarls imvarls   719 Sep 11 09:20 server.c
-rwxr-xr-x  1 imvarls imvarls 16272 Sep 11 09:14 sever
drwx----- 3 imvarls imvarls  4096 Aug 21 14:06 snap
-rwxr-xr-x  1 imvarls imvarls 16256 Aug  4 07:06 task2
-rw-r--r--  1 imvarls imvarls  1308 Aug  4 07:05 task2.c
-rwxr-xr-x  1 imvarls imvarls 16392 Aug 21 13:54 week1
-rw-r--r--  1 imvarls imvarls   680 Aug 21 13:53 week1.c
-rwxr-xr-x  1 imvarls imvarls 16288 Aug 25 06:34 week4
-rw-r--r--  1 imvarls imvarls   990 Aug 25 06:33 week4.c

--> [Parent Process] Child process has finished. Parent exiting.
imvarls@LAPTOP-VQ4D70L7: ~$

```

```

--> [Parent Process] Child process has finished. Parent exiting.
imvarls@LAPTOP-VQ4D70L7: ~$ ps -f
UID          PID     PPID  C  STIME TTY          TIME CMD
imvarls      410       409  0  02:50 pts/0    00:00:00 -bash
imvarls      620       410  0  02:52 pts/0    00:00:00 ps -f
imvarls@LAPTOP-VQ4D70L7: ~$

```

```

imvarls@LAPTOP-VQ4D78L7:~$ ps -f
UID          PID     PPID  C  STIME TTY          TIME CMD
imvarls      410       409  0   02:50 pts/0    00:00:00 -bash
imvarls      620       410  0   02:52 pts/0    00:00:00 ps -f
imvarls@LAPTOP-VQ4D78L7:~$ top
top - 02:52:17 up 2 min,  1 user,  load average: 0.14, 0.09, 0.03
Tasks:  31 total,   1 running, 30 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  7782.6 total,  6281.4 free,   566.9 used,  1086.6 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used.  7215.8 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
    86 root        20   0   35668  12640  9252  S   0.3   0.2   0:00.29 systemd-udevd
     1 root        20   0   24196  15332 11516  S   0.0   0.2   0:01.10 systemd
     2 root        20   0    3180   2208  2072  S   0.0   0.0   0:00.01 init-systemd(Ub
     7 root        20   0    3180   2040  1940  S   0.0   0.0   0:00.00 init
    47 root        19  -1   42200  16572 15308  S   0.0   0.2   0:00.19 systemd-journal
    83 systemd+    20   0   22540  14608 11984  S   0.0   0.2   0:00.08 systemd-resolve
   164 root        20   0  159348  1868   1540  S   0.0   0.0   0:00.00 snapfuse
   166 root        20   0  159348  1884   1556  S   0.0   0.0   0:00.00 snapfuse
   169 root        20   0  380412  11608  1508  S   0.0   0.1   0:02.72 snapfuse
   195 root        20   0    2888   1948  1820  S   0.0   0.0   0:00.08 chronyd-starter
   196 root        20   0    4472   3124  2880  S   0.0   0.0   0:00.00 cron
   197 message+    20   0    8796   5352  4692  S   0.0   0.1   0:00.07 dbus-daemon
   202 root        20   0   44092  29752 14616  S   0.0   0.4   0:00.38 networkd-dispat
   205 root        20   0 2073028  42004 25124  S   0.0   0.5   0:00.71 snapd
   210 root        20   0   18436   9432  8188  S   0.0   0.1   0:00.13 systemd-logind
   218 root        20   0 1866288  15356 12552  S   0.0   0.2   0:00.14 wsl-pro-service
   241 root        20   0    5492   2704  2516  S   0.0   0.0   0:00.00 agetty
   246 syslog      20   0  220548   5876  4648  S   0.0   0.1   0:00.09 rsyslogd
   263 _chrony       20   0   24248  11784  7156  S   0.0   0.1   0:00.06 chronyd
   270 root        20   0  123456  32604 17996  S   0.0   0.4   0:00.33 unattended-upgr
   283 _chrony       20   0   12092   2464  1820  S   0.0   0.0   0:00.01 chronyd
   408 root        20   0    3184   1112    980  S   0.0   0.0   0:00.00 SessionLeader
   409 root        20   0    3200   1248  1108  S   0.0   0.0   0:00.01 Relay(410)
   410 imvarls      20   0    6268   5536  3864  S   0.0   0.1   0:00.05 bash
   411 root        20   0    8648   5588  4744  S   0.0   0.1   0:00.01 login
   456 imvarls      20   0   22352  13452 10820  S   0.0   0.2   0:00.11 systemd
   458 imvarls      20   0   22908   4124  2124  S   0.0   0.1   0:00.00 (sd-pam)
   486 imvarls      20   0    6136   5416  3856  S   0.0   0.1   0:00.04 bash
   630 imvarls      20   0    8204   6052  3920  R   0.0   0.1   0:00.00 top
   631 root        20   0   35672   6812  3392  S   0.0   0.1   0:00.00 (udev-worker)
   632 root        20   0   35672   6812  3392  S   0.0   0.1   0:00.00 (udev-worker)

```

Observation

- The parent process successfully created three child processes using fork().
- Each child process executed independently and terminated after a different delay.
- The wait() function waited for any one child process to complete and returned the PID of the completed child.
- The waitpid() function waited for a specific child process using its PID
- The parent process synchronized with all child processes before terminating.
- Thus, wait() provides general child-process synchronization, while waitpid() provides more specific control over which child process waited for.

Task-2 Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1 zombie.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // Child process exits immediately
        printf("Child process started. PID = %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    } else {
        // Parent process sleeps, leaving the child in a zombie state
        printf("Parent process started. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping. Child will become a zombie.\n");
        sleep(30);
        printf("Parent process terminating.\n");
    }
}
```

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
imvarls@LAPTOP-VQ4D70L7:~$ nano zombie.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc zombie.c -o zombie
imvarls@LAPTOP-VQ4D70L7:~$ ./zombie
Parent process started. PID = 949
Child PID = 950
Parent sleeping. Child will become a zombie.
Child process started. PID = 950
Child process exiting...
```

```

Parent process terminating.
imvarls@LAPTOP-VQ4D79L7:~$ ps -el
F S  UID      PID     PPID  C  PRI  NI  ADDR  SZ  WCHAN  TTY      TIME  CMD
4 S   0         1         0  0  80   0   -  6116  -    ?        00:00:01 systemd
5 S   0         2         1  0  80   0   -   795  -   hvcs0    00:00:00 init-systemd(Ub
0 S   0         7         2  0  80   0   -   795  -   hvcs0    00:00:00 init
4 S   0        47         1  0  79  -1 - 12599  -    ?        00:00:00 systemd-journal
4 S  989        83         1  0  80   0   -  5635  -    ?        00:00:00 systemd-resolve
4 S   0        86         1  0  80   0   -  8917  -    ?        00:00:00 systemd-udev
1 S   0       164         1  0  80   0   - 39837  -    ?        00:00:00 snapfuse
1 S   0       166         1  0  80   0   - 39837  -    ?        00:00:00 snapfuse
1 S   0       169         1  0  80   0   - 95103  -    ?        00:00:02 snapfuse
4 S   0       195         1  0  80   0   -   722  -    ?        00:00:00 chronyd-starter
4 S   0       196         1  0  80   0   -  1118  -    ?        00:00:00 cron
4 S  995       197         1  0  80   0   -  2199  -    ?        00:00:00 dbus-daemon
4 S   0       202         1  0  80   0   - 11023  -    ?        00:00:00 networkd-dispat
4 S   0       205         1  0  80   0   - 536818  -    ?        00:00:02 snapd
4 S   0       210         1  0  80   0   -  4609  -    ?        00:00:00 systemd-logind
4 S   0       241         1  0  80   0   -  1373  -   tty1    00:00:00 agetty
4 S  100       246         1  0  80   0   - 55137  -    ?        00:00:00 rsyslogd
4 S  988       263        195  0  80   0   -  6062  -    ?        00:00:00 chronyd
4 S   0       270         1  0  80   0   - 30864  -    ?        00:00:00 unattended-upgr
1 S  988       283        263  0  80   0   -  3023  -    ?        00:00:00 chronyd
4 S   0       411         2  0  80   0   -  2162  -    ?        00:00:00 login
4 S 1000       456         1  0  80   0   -  5588  -    ?        00:00:00 systemd
5 S 1000       458        456  0  80   0   -  5727  -    ?        00:00:00 (sd-pam)
4 S 1000       486        411  0  80   0   -  1534  poll_s pts/1    00:00:00 bash
1 S   0       847         1  0  80   0   - 39837  -    ?        00:00:00 snapfuse
5 S   0       890         2  0  80   0   -   796  -    ?        00:00:00 SessionLeader
5 S   0       891        890  0  80   0   -   800  -    ?        00:00:00 Relay(892)
4 S 1000       892        891  0  80   0   -  1567  do_wai pts/0    00:00:00 bash
0 R 1000       960        892  0  80   0   -  1798  -   pts/0    00:00:00 ps
imvarls@LAPTOP-VQ4D79L7:~$

```

Observation

- A child process was created using `fork()`.
- The child terminated while the parent continued executing without calling `wait()` or `waitpid()`.
- The terminated child remained in the process table as a zombie (Z) /process because its exit status had not been collected by the parent.
- The program was then modified to use `waitpid()` for proper synchronization.
- After the modification, the parent collected the child's exit status immediately after its termination.
- The child was therefore removed from the process table and no zombie process was created.
- This demonstrates that proper parent-child synchronization using `wait()` or `waitpid()` prevents zombie processes.